

02-Curso BLE ESP32

nRF Connect

#ble #android #iphone #ipad

Una de las herramientas más útiles para el desarrollo de BLE es la aplicación nRF Connect de Nordic Semiconductor. Se puede encontrar para Android y para iPhone/iPad:

Google Play: https://play.google.com/store/apps/details?id=no.nordicsemi.android.mcp&hl=es_419&pli=1

App Store: <https://apps.apple.com/us/app/nrf-connect-for-mobile/id1054362403>

Página oficial de Nordic Semiconductor:

<https://www.nordicsemi.com/Products/Development-tools/nRF-Connect-for-mobile>

P01-Introducción a BLE con ESP32

#ble #esp32

En esta sección estudiaremos qué servicios son los mínimos indispensables en una comunicación usando BLE. Para ello, usaremos el código que está adjuntado (**ESP_Simple.ino**) para cargarlo en la ESP32.

 ESP_Simple.zip

Por ahora sólo nos centraremos en la función **InitBLE** del fichero:

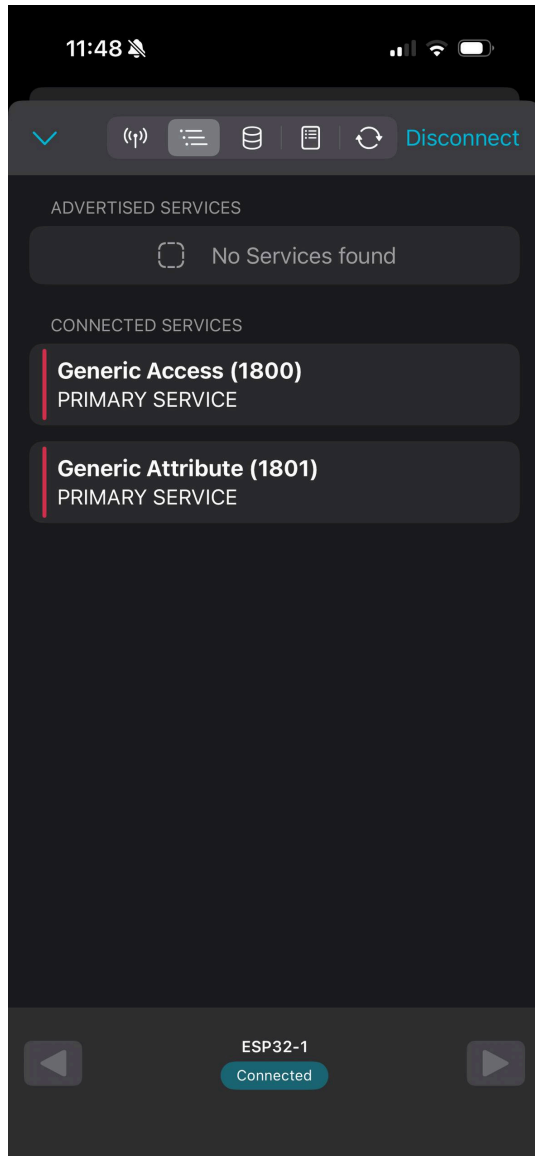


```
1 // Servidor de BLE
2 BLEServer *pServer;
3
4 // Nombre que le vamos a asignar inicialmente al dispositivo.
5 String ble_name = "ESP32-1";
6
7 [...]
8
9 void InitBLE() {
10     // Inicia el BLE y le asigna el nombre.
11     BLEDevice::init(ble_name);
12
13     // Crea el BLE Server.
14     pServer = BLEDevice::createServer();
15
16     // Inicia la publicación.
17     pServer->getAdvertising()->start();
18 }
```

La función ***init*** de la clase ***BLEDevice*** es la encargada de realizar la configuración por defecto para el dispositivo. Obviamente, dicha configuración se puede modificar, pero generalmente es suficiente con añadirle los servicios y las características que queramos enviar. Cabe destacar que el nombre que se le quiere asignar al dispositivo se puede añadir en este momento, con un ***String*** como parámetro.

Con la siguiente línea (16) se crea un servidor BLE, y posteriormente (línea 19) le indicamos que se publicite.

Una vez tengamos el fichero cargado en la ESP32, abrimos [nRF Connect](#), buscamos el dispositivo, y lo conectamos. Tras ello, nos aparecerá la siguiente información:



BLEDevice::init(ble_name) ha creado dos servicios. Las UUID de dichos servicios pertenecen a una UUID de 4 bytes, aunque dependiendo del dispositivo con el que lo estés leyendo aparezca la adaptación a 16 bytes de las UUID, en ese caso es suficiente con leer los números hasta el primer guion. Estos servicios son el **0x1800** y el **0x1801**, que corresponden a ***Generic Access***^[1] y a ***Generic Attribute***^[2] respectivamente. Si accedemos a través del enlace de *UUID de servicios* y pulsamos en los correspondientes servicios, podemos ver qué características son obligatorias,

recomendadas y opcionales, además de comprobar si deben ser sólo de lectura, sólo de escritura o lectura y escritura.

Vamos a ver las características de **Generic Access (0x1800)**:

- **Device Name**: Es obligatoria, con lectura también obligatoria, escritura opcional y sin notificaciones.
- **Appearance**: Es obligatoria, con lectura obligatoria, y escritura y notificaciones prohibidas.
- **Peripheral Privacy Flag**: Opcional si el servidor sólo admite un enlace, obligatoria en otro caso. La lectura obligatoria y la escritura opcional si sólo admite un enlace y obligatoria en otro caso. Tampoco permite notificaciones.
- **Reconnection Address**: Es condicional, y está subordinada a la implementación de **Peripheral Privacy Flag**. Es una característica que sólo admite escritura y no permite notificaciones.
- **Peripheral Preferred Connection Parameters**: También es opcional, de sólo lectura y sin notificaciones.

Como podemos observar, el código que hemos ejecutado sólo implementa las dos obligatorias de este servicio, además de una extra, que no usaremos en este curso.

En cuanto a **Generic Attribute (0x1801)**, podemos comprobar que únicamente tiene una característica, **Service Changed**, que además es opcional, con lectura, escritura y notificación excluida, pero con indicación obligatoria. Además tiene un descriptor (**Client Characteristic Configuration**) que es obligatorio, con lectura y escritura también obligatoria; cuya función es asignar un identificador al cliente cuando el servidor permite múltiples enlaces.

-
1. <https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/Core-54/out/en/host/generic-access-profile.html> ↩
 2. <https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/Core-54/out/en/host/generic-attribute-profile--gatt-.html> ↩

P02-Modificando servicios y añadiendo Device Information

#ble

#esp32

#arduino

Tras el estudio de los servicios más básicos, sus características y sus descriptores, procederemos a modificar el código para permitir cambiar el nombre del dispositivo en el servicio **Generic Access (0x1800)** y añadir el servicio **Device Information (0x180A)*^[1].

 ESP_Simple_2.zip

Comenzaremos con el código necesario para modificar las propiedades de Generic Access:



```
1 // UUIDs de los servicios.
2 #define GENERIC_ACCESS_UUID "00001800-0000-1000-8000-00805f9b34fb"
3
4 // Característica Device Name
5 BLECharacteristic DeviceNameCharacteristic(
6     BLEUUID((uint16_t)0x2A00),
7     BLECharacteristic::PROPERTY_READ |
8     BLECharacteristic::PROPERTY_WRITE
9 );
10
11 [...]
12
13 void InitBLE() {
14     [...]
15
16     // Crea el servicio Generic Access
17     pGeneralAccess = pServer->createService(GENERIC_ACCESS_UUID);
18     // Añade la característica del Generic Access
19     pGeneralAccess->addCharacteristic(&DeviceNameCharacteristic);
20     // Cambia el nombre - similar a BLEDevice::init(ble_name), si no
21     // sustituyésemos la característica.
22     DeviceNameCharacteristic.setValue("ESP32-2");
23
24     // Añade el servicio
25     pServer->getAdvertising()->addServiceUUID(GENERIC_ACCESS_UUID);
26
27     // Inicia el servicio
28     pGeneralAccess->start();
29
30     [...]
31 }
```

En las líneas 7 y 8 podemos apreciar las propiedades que le damos a la característica, en concreto, añadimos la propiedad de escritura para poder modificar el nombre. Realmente, nada impide poner más propiedades, como por ejemplo `BLECharacteristic::PROPERTY_NOTIFY`, sin embargo, podemos producir un funcionamiento no deseado en el receptor si no se cumple apropiadamente la normativa.

En la línea 17 estamos creando un nuevo servicio que sobrescribirá al que existe por defecto, ya que si intentamos acceder a este último para modificarlo, la ESP32 dará un error y se reiniciará. Al nuevo servicio de *Generic Access* le añadimos la característica *Device Name*, y cambiamos el nombre a ESP32-2 para comprobar cuál prevalece. Finalmente, añadimos el servicio a las publicaciones, y lo iniciamos.

Ahora nos centraremos en añadir el servicio **Device Information (0x180A)**^[1-1]. Antes de nada debemos mirar la normativa para conocer las características de las que dispone, y con qué configuración hay que implementarlas. Lo primero que observamos es que tiene nueve posibles características, que todas son opcionales y exclusivamente de lectura, sin notificaciones. Si nos fijamos en los nombres de las características, vemos que se corresponden con las típicas para describir un dispositivo, como por ejemplo, fabricante, modelo, número de serie, versión de *hardware*, de *firmware* y de *software*, etc. Cabe destacar que, en producción en serie, **PnP ID** se puede usar para asignar un identificador único al dispositivo. Para el ejemplo que estamos realizando, sólo implementaremos **Manufacturer Name String**:



```
1  #define DEVICE_INFORMATION_UUID "0000180a-0000-1000-8000-00805f9b34fb"
2
3  // Característica Manufacturer Name String
4  BLECharacteristic ManufacturerNameCharacteristic(
5      BLEUUID((uint16_t)0x2A29),
6      BLECharacteristic::PROPERTY_READ
7  );
8
9  // Servicio Device Information
10 BLEService *pDeviceInfo;
11
12 [...]
13
14 void InitBLE() {
15     [...]
16     // Crea el servicio Device Name
17     pDeviceInfo = pServer->createService(DEVICE_INFORMATION_UUID);
18     // Añade la característica Manufacturer Name String
19     pDeviceInfo->addCharacteristic(&ManufacturerNameCharacteristic);
20     // Añade el nombre del fabricante
21     ManufacturerNameCharacteristic.setValue(
22         "Departamento de Tecnología Electrónica");
23
24     // Añade el servicio Generic Access
25     pServer->getAdvertising()->addServiceUUID(DEVICE_INFORMATION_UUID)
26
27     // Inicia el servicio
28     pDeviceInfo->start();
29
30     [...]
31 }
```

Puesto que Manufacturer Name String no tiene ningún descriptor asociado según las especificaciones de la característica en www.bluetooth.com, sino que únicamente tiene un valor, es lo que añadimos en el código superior (línea 21). Posteriormente, añadimos

el servicio a la publicación (línea 25), y lo iniciamos (línea 28), tal y como se hizo en el caso anterior.

En el fichero v2 se encuentra la solución para evitar tener dos GAP en paralelo.

#TODO : COMPROBAR.

 ESP_Simple_2_v2.zip

1. https://www.bluetooth-com.translate.goog/specifications/specs/device-information-service/?_x_tr_sl=auto&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=wapp ↩ ↩

P03-Envío de notificaciones. Battery Service.

#ble

#esp32

#arduino

En el módulo anterior se añadió un servicio que era sólo de lectura y modificamos una característica para que fuese de escritura. En esta ocasión crearemos un servicio que informará del porcentaje de batería que le queda al dispositivo vía notificaciones. Los valores que se enviarán por el servicio serán simulados, para poder comprobar que se envían correctamente las notificaciones sin esperar el tiempo de carga/descarga.

 ESP_Simple_3.zip

Lo primero que tenemos que hacer es buscar la normativa correspondiente a este servicio en www.bluetooth.com. El servicio se llama **Battery Service**, y posee una única característica **Battery Level**, cuya lectura es obligatoria y la notificación es opcional, también uno de los descriptores es obligatorio ya que indica el formato del valor que se envía. Si miramos el GATT de la característica podemos observar que sólo posee un campo que es el de valor. Por tanto para configurar este servicio necesitamos el siguiente código:



```
1  #define BATTERY_SERVICE_UUID "0000180f-0000-1000-8000-00805f9b34fb"
2
3  // Característica Battery Level
4  BLECharacteristic BatteryLevelCharacteristic(
5      BLEUUID((uint16_t)0x2A19),
6      BLECharacteristic::PROPERTY_READ |
7      BLECharacteristic::PROPERTY_NOTIFY
8  );
9
10 // Servicio de Battery
11 BLEService *pBatteryService;
12
```

```

13 void InitBLE() {
14     [...]
15     // Crea el servicio de Battery
16     pBatteryService = pServer->createService(BATTERY_SERVICE_UUID);
17     // Añade la característica del nivel de batería.
18     pBatteryService->addCharacteristic(&BatteryLevelCharacteristic);
19     // Añade las notificaciones.
20     BatteryLevelCharacteristic.addDescriptor(new BLE2902());
21     // Configuración de un descriptor de formato
22     BLE2904* batteryLevelDescriptor = new BLE2904();
23     batteryLevelDescriptor->setFormat(BLE2904::FORMAT_UINT8);
24     batteryLevelDescriptor->setNamespace(1);
25     // Esta UUID indica que es un porcentaje.
26     batteryLevelDescriptor->setUnit(0x27ad);
27     // Añade el descriptor.
28     BatteryLevelCharacteristic.addDescriptor(batteryLevelDescriptor);
29     // Añade el servicio Battery
30     pServer->getAdvertising()->addServiceUUID(BATTERY_SERVICE_UUID);
31
32     // Inicia el servicio
33     pBatteryService->start();
34
35     [...]
36 }

```

Como se puede observar, el código es parecido a lo que ya vimos en el módulo anterior, aunque aparecen dos nuevos descriptors, el **BLE2902** y el **BLE2904**^[1]. El primero de ellos se encarga de realizar las notificaciones, y el segundo, es el que indica el formato del valor que se está enviando. Sin embargo, no estamos enviando información de la batería. Para ello, añadiremos el siguiente código al **loop**:

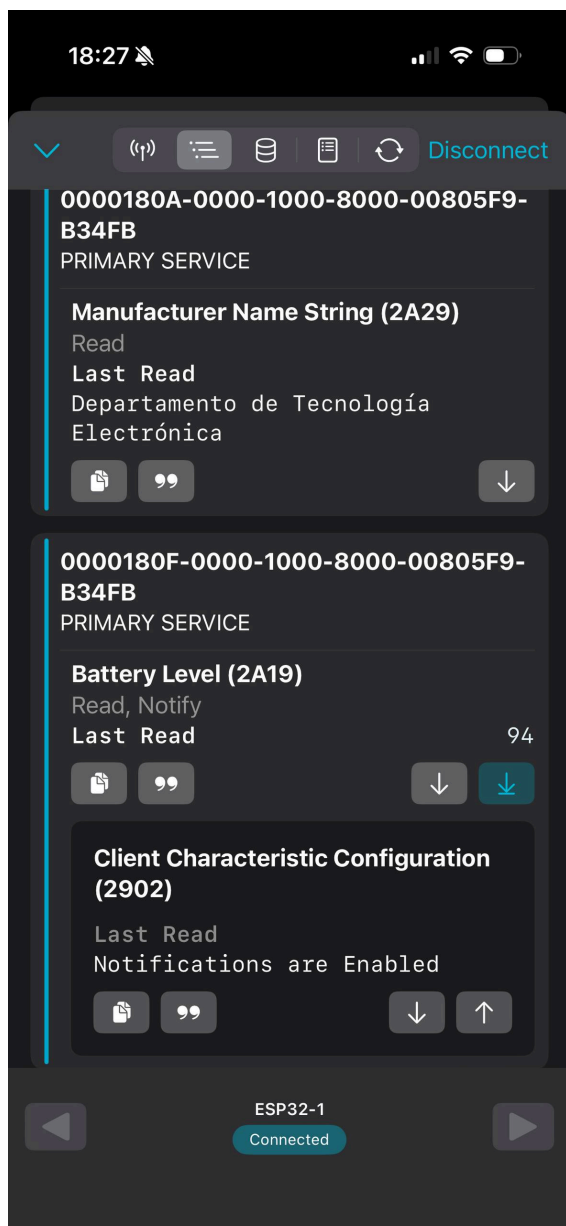


```

1 void loop() {
2     [...]
3     // Formato usado (hay otros): setValue(uint8_t *, size)
4     BatteryLevelCharacteristic.setValue(&level, 1);
5     // Se activa la notificación de nuevo valor.
6     BatteryLevelCharacteristic.notify(true);
7     [...]
8 }

```

En la línea 4 se envía el valor, y en la 6 la notificación. El true es para forzar a que se envíe la notificación. En la siguiente imagen podemos ver el servicio usando la aplicación nRF Connect:



1. <https://www.bluetooth.com/specifications/assigned-numbers/> ↩

P04 - Servicio complejo - Ritmo Cardíaco

#ble

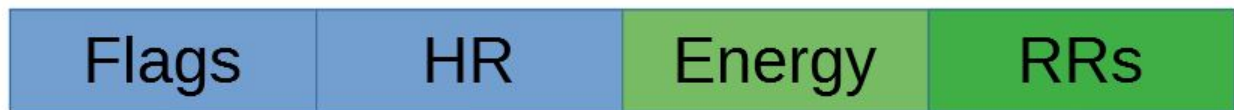
#esp32

#arduino

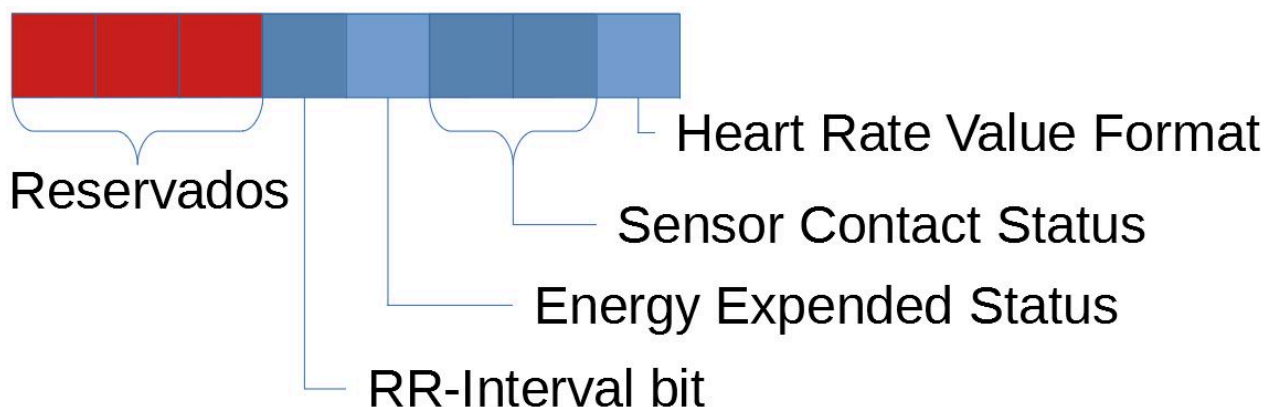
Como ejemplo de un servicio más complejo implementaremos el ritmo cardíaco, y para ello usaremos el servicio **Heart Rate (0x180D)**^{[1][2]}. Si leemos la normativa relacionada con este servicio, podemos ver que tiene tres características: **Heart Rate Measurement** que es obligatoria y como única propiedad tiene la notificación como obligatoria; **Body Sensor Location**, que es opcional, y si se implementa la única propiedad es lectura de forma obligatoria; y **Heart Rate Control Point**, que es condicional, pero obligatoria si el dispositivo ofrece el gasto energético. Para el ejemplo configuraremos las dos primeras características únicamente. Por ello, de los cuatro requerimientos del servicio, tenemos que configurar tres: **Notifications**, **Read Characteristic Descriptor** y **Write Characteristic Descriptors**. En este caso sí sería

interesante consultar el documento de la normativa, que se adjunta como recurso, en concreto se deben consultar la **HRP** y la **HRS**.

Heart Rate Measurement tiene como UUID **0x2A37**, y es la primera característica "especial" que nos vamos a encontrar: sólo tiene un valor, pero que a su vez posee distintos campos, en los que no todos tienen por qué estar a la vez. ¿Cómo se resuelve a la hora del envío? La solución es fácil: enviarlo como un array de unsigned int de tamaño byte. En la siguiente figura se muestra la estructura de los campos, con fondo verde están los campos opcionales.



- **Flags**: es de tamaño byte, y tiene cuatro campos útiles. El primero de ellos es **Heart Rate Value Format**, e indica el tamaño que tiene el campo HR, si es de 8 bits el flag está desactivado; si es de 16 bits, el flag vale 1. Cuando el flag HRVF está desactivado, el valor máximo que se puede mandar de heart rate es 255 latidos por minuto (bpm). El siguiente campo es **Sensor Contact Status**, que indica si el dispositivo tiene conocimiento de si está colocado (segundo bit del campo, tercero de Flags), y en caso de que lo tenga, si detecta el contacto o no (primer bit del campo, segundo de Flags). El siguiente es **Energy Expended Status**, que indica si se contabiliza el gasto energético (medido en kilojulios) o no. Finalmente tenemos **RR-Interval**, un bit que indica si se envían los intervalos RR, es decir el tiempo que pasa entre dos latidos consecutivos.



- **Heart Rate Measurement Value**: En éste se envía el ritmo cardíaco en latidos por minuto y, junto a Flags, es el único obligatorio. Su tamaño puede ser de un byte o dos bytes, dependiendo del valor que hubiésemos asignado a HRVF, y siempre es unsigned int. Cuando está activo suele ser porque el dispositivo esté destinado a medir el ritmo cardíaco en animales, ya que con 255 latidos por minuto es suficiente hasta para detectar taquicardia supraventricular en humanos.
- **Energy Expended**: Sólo se envía si está activo el flag asociado. Además, activar dicho flag también hace que sea obligatorio activar la característica **Heart Rate Control Point**, ya que es con la que se resetea el valor de energía cuando alcanza el máximo (ver especificación HRS apartado 3.1.1.3). Cuando se envía, tiene el

formato de UINT16. También se recomienda enviar cada vez que se actualice el HRMV. No se implementa en este ejemplo.

- **RR Interval Field:** Sólo se envía si está activo el flag asociado. Se pueden enviar un máximo de 9 intervalos RR en formato UINT16, restándole uno al máximo si se envía el gasto energético y otro adicional si HRMV tiene formato UINT16. Si hay más del máximo permitido que enviar, se guardan para la siguiente actualización. La unidad de este campo es segundos con una resolución de 1/1024. Tampoco se implementa en este ejemplo.

Finalmente habría que tener en cuenta el apartado 3.1.1.5 del documento HRS, ya que nos informa que el HRMV se suele actualizar una vez por segundo, enviando los intervalos RR si se ha producido alguno. Por otro lado el gasto energético se suele actualizar cada 10 segundos aproximadamente, si está implementado. Estos intervalos, en todo caso, son decididos por el servidor y el cliente no puede modificarlos.

Body Sensor Location tiene como UUID **0x2A38** y es la última característica que implementaremos en el ejemplo. Esta característica y sólo tiene un campo, con una longitud de byte, y en la que se indica la posición del cuerpo en la que se encuentra el sensor. Le asignaremos el valor 2, que significa que se colocará en la muñeca. Recordemos que sólo es obligatoria la lectura.

Para implementar ambas características del servicio, necesitaremos añadir código en el de ES_Simple_3.



CharacteristicsAndInitBLE

```
1  #define HR_UUID "0000180d-0000-1000-8000-00805f9b34fb"
2
3  // Característica Heart Rate Measurement
4  BLECharacteristic HRMCharacteristic(
5      BLEUUID((uint16_t)0x2A37),
6      BLECharacteristic::PROPERTY_NOTIFY
7  );
8
9  // Característica Body Sensor Location
10 BLECharacteristic BSLCharacteristic(
11     BLEUUID((uint16_t)0x2A38),
12     BLECharacteristic::PROPERTY_READ
13 );
14
15 // Servicio de Heart Rate
16 BLEService *pHRService;
17
18 void InitBLE() {
19     [...]
20     // Crea el servicio de Heart Rate Measurement
21     pHRService = pServer->createService(HR_UUID);
```

```

22 // Añade la característica del HRM
23 pHRService->addCharacteristic(&HRMCharacteristic);
24 // Añade las notificaciones a HRM.
25 HRMCharacteristic.addDescriptor(new BLE2902());
26 // Añade la característica del BSL
27 pHRService->addCharacteristic(&BSLCharacteristic);
28 // Indica que la posición es en la muñeca.
29 uint8_t aux = 2;
30 BSLCharacteristic.setValue(&aux, 1);
31
32 // Añade el servicio Heart Rate
33 pServer->getAdvertising()->addServiceUUID(HR_UUID);
34
35 // Inicia el servicio
36 pHRService->start();
37 [...]
38 }

```



loop()

```

1 void loop() {
2 // Array con distintos valores de HR en latidos por minuto (bpm).
3 static uint8_t ahr[7] = {60, 61, 62, 63, 70, 68, 66};
4 // Contador para movernos circularmente en el array anterior
5 static uint8_t cont = 0;
6 // Estructura de datos para enviar el HR por el servicio apropiado.
7 // 6 indica no intervalo RR, no gasto energético, contacto detectado
8 // ritmo cardíaco en tamaño byte.
9 static uint8_t msg[2] = {6, 0};
10
11 [...]
12
13 // Se añade el valor seleccionado de HR.
14 msg[1] = ahr[cont];
15 // Se añade el valor a la característica.
16 // En este caso, HR, contacto detectado.
17 // Formato usado (hay otros): setValue(uint8_t *, size)
18 HRMCharacteristic.setValue(msg, 2);
19 // Se activa la notificación de nuevo valor.
20 HRMCharacteristic.notify(true);
21
22 [...]
23 }

```

Devices

DISCONNECT

BONDED

ADVERTISER

ESP32-1
24:6F:28:B0:87:AE

X

CONNECTED

BONDED

CLIENT

SERVER

UUID: 0x1800

PRIMARY SERVICE

Device Information

UUID: 0x180A

PRIMARY SERVICE

Battery Service

UUID: 0x180F

PRIMARY SERVICE

Heart Rate

UUID: 0x180D

PRIMARY SERVICE

Body Sensor Location

UUID: 0x2A38

Properties: READ

Value: Wrist

Heart Rate Measurement

UUID: 0x2A37

Properties: NOTIFY

Value: Heart Rate Measurement: 68 bpm,
Contact is Detected

Descriptors:

Client Characteristic Configuration

UUID: 0x2902

Value: Notifications enabled

1. https://www.bluetooth-com.translate.google/specifications/specs/heart-rate-profile-1-0/?x_tr_sl=auto&x_tr_tl=es&x_tr_hl=es&x_tr_pto=wapp ↩
2. https://www.bluetooth-com.translate.google/specifications/specs/heart-rate-service-1-0/?x_tr_sl=auto&x_tr_tl=es&x_tr_hl=es&x_tr_pto=wapp ↩

P05-CheckClients

#ble

#esp32

#arduino

Otro elemento interesante es comprobar si existe un cliente antes de enviar la información, ya que de esa manera, ahorramos batería. De camino solucionaremos un problema aún no descubierto: ¿Qué pasa cuando el cliente se desconecta y quiere volver a reconectarse? La respuesta con el código que teníamos en las versiones anteriores del Servicio HRM es que no enviaba la información.

Necesitamos añadir el siguiente código para comprobar si existe un cliente, y para permitir la reconexión:



loop()

```

1  if (pServer->getConnectedCount() > 0) {
2      // Código de gestión del envío.
3      Serial.println("Cliente conectado");
4  } else {
5      Serial.println("Esperando cliente");
6      pServer->getAdvertising()->start();
7  }

```

Fichero:

ESP_Completo_CheckClient.zip

P06- Otros aspectos

#ble

#esp32

#arduino

Hasta ahora hemos aprendido a crear un servidor BLE con ESP32, añadir servicios, características y descriptores, buscar en fuentes fiables la documentación necesaria para su implementación... Sin embargo, aún quedan más aspectos a explorar en una conexión BLE. Por ejemplo, en el *loop* siempre estamos enviando datos, sin comprobar si el dispositivo tiene un cliente conectado, aumentando así el gasto energético en un estándar que está pensado para reducirlo al mínimo. ¿Podríamos evitar este aspecto? La respuesta, obviamente es que sí, mediante el uso de los *Callbacks*: tanto la clase **BLEServer**, como **BLECharacteristic** lo tienen.

El *Callback* de **BLEServer** posee cuatro métodos virtuales en su clase **BLEServerCallbacks** que se deben implementar en la clase que herede: el destructor, **onConnect(...)** y **onDisconnect(...)**. A estos métodos le podemos añadir que controlen una variable global para saber si han sido invocados, además de parar los servicios en **onDisconnect** si así lo deseamos. En el ejemplo que está en *Archivo/Ejemplos/ESP32 BLE Arduino/BLE_notify* encontramos una implementación sencilla.

En cuanto al *Callback* de la clase **BLECharacteristicCallbacks**, perteneciente a **BLECharacteristic**, tiene 5 métodos virtuales: el destructor, **onRead(...)**, **onWrite(...)**, **onNotify(...)**, y **onStatus(...)**. En el ejemplo que está en *Archivo/Ejemplos/ESP32 BLE Arduino/BLE_uart* encontramos una implementación sencilla.

También queda por ver la seguridad, aceptar múltiples clientes, y otros contenidos que se escapan al objetivo del presente curso. Sin embargo, antes de cerrar esta sección, nos gustaría recomendar una página con una gran cantidad de contenido sobre este dispositivo: <http://esp32.net/> .